React — Zero to Master

23 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction and Setup
02 JSX
03 Components and Props
04 State with useState
05 Events and Conditional Rendering
06 Lists and Keys
07 Forms and Controlled Inputs

Intermediate (6 topics)

08 useEffect and Side Effects
09 Data Fetching
10 useRef, useMemo, useCallback
11 Context API
12 React Router
13 Custom Hooks

Advanced (6 topics)

14 useReducer and State Patterns
15 Global State: Redux Toolkit, Zustand
16 Performance Optimisation
17 Forms at Scale
18 Server State with TanStack Query
19 Testing React

Master (4 topics)

20	React 19 Features
21	Project Architecture and Best Practices
22	Real Project: E-commerce Front End
23	Deployment and Interview Prep

Beginner Level

01. Introduction and Setup

DEFINITION

React is a JavaScript library for building UIs from components. Create an app with Vite.

SYNTAX

```
npm create vite@latest
```

EXAMPLE

```
npm create vite@latest my-app -- --template react-ts
cd my-app && npm install && npm run dev
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop display built from ready-made racks; React builds pages from reusable components.

தமிழ்

தயார் செய்யப்பட்ட அலமாரிகளால் கடை காட்சி அமைப்பது போல React மறுபயன்பாட்டு components கொண்டு பக்கங்களை உருவாக்குகிறது.

PROS

- Huge ecosystem and community
- Component reuse
- Flexible

CONS

- Only a view library (you choose router/state)
- Fast-changing best practices

ADVANCED / NEW FEATURES

- React 19 and the React Compiler
- Vite is the standard starter; Create React App is deprecated

02. JSX

DEFINITION

JSX looks like HTML inside JavaScript. Use `{}` for expressions, `className` instead of `class`, and return one root element.

SYNTAX

```
<div className="box">{value}</div>
```

EXAMPLE

```
const App = () => <h1 className="title">Hello {name}</h1>;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A wedding invitation template: fixed design with blanks for names and dates; JSX mixes markup with values.

தமிழ்

திருமண அழைப்பிதழ் மாதிரியில் நிலையான வடிவமைப்பும் பெயர், தேதிக்கான வெற்றிடமும் இருப்பது போல JSX markup-உடன் மதிப்புகளைக் கலக்கிறது.

PROS

- Readable UI code
- Full JavaScript power

CONS

- Mixing logic and markup confuses some beginners
- Needs a build step

ADVANCED / NEW FEATURES

- React 19: `ref` as a prop, document metadata tags (`<title>`) in components

03. Components and Props

DEFINITION

A component is a function returning JSX. Props pass data from parent to child and are read-only.

SYNTAX

```
function Card({ title }) { return <h2>{title}</h2>; }
```

EXAMPLE

```
function Card({ title }: { title: string }) { return <h2>{title}</h2>; }
<Card title="React" />
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A pamphlet printing machine: the same layout with a different heading for each pamphlet; props change the heading.

தமிழ்

ஒரே வடிவமைப்பில் ஒவ்வொரு துண்டுப் பிரசுரத்துக்கும் வேறு தலைப்பு அச்சிடும் இயந்திரம் போல props தலைப்பை மாற்றுகின்றன.

PROS

- Reusable and composable
- Easy testing

CONS

- Prop drilling in deep trees

ADVANCED / NEW FEATURES

- Server Components (React 19)
- `ref` as prop removes `forwardRef`

04. State with useState

DEFINITION

State is data that changes over time; updating it re-renders the component.

SYNTAX

```
const [state, setState] = useState(initial);
```

EXAMPLE

```
const [count, setCount] = useState(0);
<button onClick={() => setCount(c => c + 1)}>{count}</button>
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A scoreboard at a kabaddi match: when the score changes, the board redraws. State changes trigger a re-render.

தமிழ்

கபடி போட்டி ஸ்கோர் போர்டில் மதிப்பெண் மாறினால் போர்டு மறுபடி வரையப்படுவது போல state மாறினால் மீண்டும் வரையப்படும்.

PROS

- Simple mental model
- Local component data

CONS

- Stale closure bugs

- Updates are batched and asynchronous

ADVANCED / NEW FEATURES

- Automatic batching (18)
- `useOptimistic` and `useActionState` (19)

05. Events and Conditional Rendering

DEFINITION

Handle events with camelCase props; render conditionally with `&&`, ternary or early return.

SYNTAX

```
<button onClick={handler}>OK</button>
{cond ? <A/> : <B/>}
```

EXAMPLE

```
{loggedIn ? <Dashboard /> : <Login />}
{items.length > 0 && <List items={items} />}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bell rings (event) and the class changes subject; conditional rendering shows 'open' or 'closed' board.

தமிழ்

மணி அடித்தால் (event) பாடம் மாறுவது போல; conditional rendering 'திறந்துள்ளது/மூடப்பட்டுள்ளது' பலகையைக் காட்டும்.

PROS

- Declarative UI
- Plain JavaScript conditions

CONS

- Inline functions can cause extra renders
- Long ternaries are hard to read

ADVANCED / NEW FEATURES

- React Compiler auto-memoises handlers

06. Lists and Keys

DEFINITION

Render arrays with `map` and give each item a stable unique `key`.

SYNTAX

```
{items.map(i => <li key={i.id}>{i.name}</li>)}
```

EXAMPLE

```
{todos.map(t => <li key={t.id}>{t.title}</li>)}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Each student in a roll call has a unique roll number (key) so the teacher can track attendance correctly.

தமிழ்

வருகைப் பதிவில் ஒவ்வொரு மாணவருக்கும் தனித்துவ பதிவு எண் (key) இருப்பது போல list உருப்படிகளுக்கு key இருக்க வேண்டும்.

PROS

- Efficient list updates
- Simple rendering

CONS

- Index keys cause bugs
- Missing keys hurt performance

ADVANCED / NEW FEATURES

- React DevTools highlight key warnings

07. Forms and Controlled Inputs

DEFINITION

A controlled input's value comes from state and changes through `onChange`.

SYNTAX

```
<input value={v} onChange={e => setV(e.target.value)} />
```

EXAMPLE

```
<input value={name} onChange={e => setName(e.target.value)} />
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A form where the clerk copies each letter you write into the register at the same time; a controlled input mirrors state.

தமிழ்

நீங்கள் எழுதும் ஒவ்வொரு எழுத்தையும் எழுத்தர் அதே நேரத்தில் பதிவேட்டில் நகலெடுப்பது போல controlled input state-ஐப் பிரதிபலிக்கும்.

PROS

- Single source of truth
- Easy validation

CONS

- Re-render per keystroke
- Boilerplate for big forms

ADVANCED / NEW FEATURES

- `useActionState` and form `action` (19)
- React Hook Form for scale

Intermediate Level

08. useEffect and Side Effects

DEFINITION

`useEffect` runs after render for fetching, subscriptions and timers; return a cleanup function; the dependency array controls when it re-runs.

SYNTAX

```
useEffect(() => { /* effect */ return () => { /* cleanup */ }; }, [deps]);
```

EXAMPLE

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);
}, []);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Switching on the fan when it gets hot (effect) and switching it off when you leave (cleanup).

தமிழ்

வெப்பமானால் மின்விசிறியை இயக்குவதும் (effect) வெளியேறும்போது அணைப்பதும் (cleanup) போல.

PROS

- Handles side effects
- Cleanup support

CONS

- Dependency mistakes cause loops/bugs
- Overused for derived state

ADVANCED / NEW FEATURES

- `useEffectEvent` (19.2)
- Effects run twice in dev Strict Mode to find bugs

09. Data Fetching

DEFINITION

Fetch in effects (or use TanStack Query) and track loading and error states; handle race conditions with cleanup/abort.

SYNTAX

```
useEffect(() => { fetch(url).then(...) }, [url]);
```

EXAMPLE

```
useEffect(() => {
  fetch('/api/users').then(r => r.json()).then(setUsers);
}, []);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Ordering from a kitchen and showing 'preparing...' until it arrives; data fetching shows loading, success and error.

தமிழ்

சமையலறைக்கு ஆர்டர் கொடுத்து தயாராகும் வரை 'தயாராகிறது...' காட்டுவது போல data fetching loading, success, error நிலைகளைக் காட்டும்.

PROS

- Flexible
- Libraries add caching

CONS

- Race conditions
- Waterfalls if careless

ADVANCED / NEW FEATURES

- `use()` hook with Suspense (19)
- TanStack Query and Server Components

10. useRef, useMemo, useCallback

DEFINITION

`useRef` holds mutable values/DOM nodes without re-rendering; `useMemo` and `useCallback` cache values and functions between renders.

SYNTAX

```
const ref = useRef(null);
const v = useMemo(() => calc(a), [a]);
```

EXAMPLE

```
const inputRef = useRef<HTMLInputElement>(null);
const total = useMemo(() => sum(items), [items]);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A sticky note on the fridge that does not change the dish (ref) and a notebook of already-calculated bills (memo).

தமிழ்

சமையலை மாற்றாத குறிப்புச் சீட்டு (ref), ஏற்கனவே கணக்கிட்ட பில்களின் குறிப்பேடு (memo) போல.

PROS

- Avoid recomputation
- Direct DOM access

CONS

- Premature memoisation adds noise
- Stale dependencies

ADVANCED / NEW FEATURES

- React Compiler memoises automatically, reducing manual `useMemo` / `useCallback`

11. Context API

DEFINITION

Share data (theme, user) across the tree without prop drilling.

SYNTAX

```
const Ctx = createContext(v);
const x = useContext(Ctx);
```

EXAMPLE

```
const ThemeContext = createContext('light');
const theme = useContext(ThemeContext);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A school announcement over the PA system reaches every class without passing a note through each room.

தமிழ்

பள்ளி ஒலிபெருக்கி அறிவிப்பு ஒவ்வொரு வகுப்புக்கும் சீட்டு கொடுக்காமலே சென்றடைவது போல Context தரவை எல்லா components-க்கும் தருகிறது.

PROS

- Avoids prop drilling
- Built in

CONS

- All consumers re-render on change
- Not for frequently updating data

ADVANCED / NEW FEATURES

- React 19: `<Context>` can be used directly as provider

12. React Router

DEFINITION

Client-side routing with `createBrowserRouter`, `Link`, nested routes, params and loaders.

SYNTAX

```
<Route path="/a/:id" element={<A />} />
```

EXAMPLE

```
<Route path="/users/:id" element={<User />} />
const { id } = useParams();
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Bus route boards: each route number leads to a different bus without leaving the bus stand.

தமிழ்

பேருந்து வழித்தட பலகைகள்: ஒவ்வொரு எண்ணும் நிலையத்தை விட்டு வெளியேறாமல் வெவ்வேறு பேருந்துக்கு அழைத்துச் செல்லும்.

PROS

- Client-side navigation
- Nested routes, loaders

CONS

- Extra library
- Version differences

ADVANCED / NEW FEATURES

- React Router v7 merges with Remix (framework mode)
- Data routers with loaders/actions

13. Custom Hooks

DEFINITION

Extract reusable stateful logic into functions starting with `use`.

SYNTAX

```
function useThing() { const [x, setX] = useState(); return [x, setX]; }
```

EXAMPLE

```
function useToggle(initial = false) {
  const [on, set] = useState(initial);
  return [on, () => set(v => !v)] as const;
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A reusable recipe card that many cooks follow; a custom hook packages repeated logic.

தமிழ்

பல சமையல்காரர்கள் பின்பற்றும் செய்முறை அட்டை போல custom hook மீண்டும் வரும் தர்க்கத்தை தொகுக்கிறது.

PROS

- Reuse logic without wrappers
- Easy to test

CONS

- Rules of hooks must be followed
- Poor naming hurts clarity

ADVANCED / NEW FEATURES

- React Compiler optimises hooks automatically

Advanced Level

14. useReducer and State Patterns

DEFINITION

Use `useReducer` for complex state transitions; lift state up and keep state minimal and derived.

SYNTAX

```
const [s, dispatch] = useReducer(reducer, init);
```

EXAMPLE

```
const [state, dispatch] = useReducer(reducer, { count: 0 });
dispatch({ type: 'inc' });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bank passbook: each entry is an action (deposit, withdraw) and the balance is computed by fixed rules.

தமிழ்

வங்கி பாஸ்புக்கில் ஒவ்வொரு பதிவும் (வரவு, செலவு) ஒரு action; இருப்பு நிலையான விதிகளால் கணக்கிடப்படும்.

PROS

- Predictable transitions
- Centralised logic

CONS

- More boilerplate than useState

ADVANCED / NEW FEATURES

- Pairs well with `use` and Server Actions

15. Global State: Redux Toolkit, Zustand

DEFINITION

For app-wide state use Redux Toolkit (slices, thunks) or lightweight Zustand stores.

SYNTAX

```
const useStore = create(set => ({ ... }));
```

EXAMPLE

```
const useStore = create<{ n: number; inc: () => void }>(set => ({ n: 0, inc: () => set(s => ({
n: s.n + 1 })) }));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

One common stock register for all branches of a chain store; a global store shares state across the app.

தமிழ்

ஒரு சங்கிலிக் கடையின் அனைத்துக் கிளைகளுக்கும் ஒரே பொது இருப்புப் பதிவேடு போல global store செயலி முழுவதும் state பகிர்கிறது.

PROS

- Shared state without prop drilling
- Devtools support

CONS

- Overkill for small apps
- Choice fatigue

ADVANCED / NEW FEATURES

- Redux Toolkit is standard Redux
- Zustand, Jotai are lightweight options

16. Performance Optimisation

DEFINITION

`React.memo`, code splitting with `lazy` / `Suspense`, list virtualisation, avoiding unnecessary renders; measure with React DevTools Profiler.

SYNTAX

```
const Comp = lazy(() => import('./Comp'));
<Suspense fallback={...}>
```

EXAMPLE

```
const Chart = lazy(() => import('./Chart'));
<Suspense fallback={<p>Loading</p>}><Chart /></Suspense>
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Opening extra counters only when the crowd grows; code splitting loads heavy parts only when needed.

தமிழ்

கூட்டம் அதிகமாகும்போது மட்டும் கூடுதல் கவுண்டர்களைத் திறப்பது போல code splitting கனமான பகுதிகளை தேவைப்படும்போது மட்டும் ஏற்றும்.

PROS

- Faster loads
- Smooth UX with Suspense

CONS

- Needs careful fallbacks
- Over-splitting hurts

ADVANCED / NEW FEATURES

- React Compiler (1.0)
- Concurrent features: `useTransition`, `useDeferredValue`
- `<Activity>` (19.2)

17. Forms at Scale

DEFINITION

Use React Hook Form with Zod for validation, performant uncontrolled inputs and typed schemas.

SYNTAX

```
const { register, handleSubmit } = useForm();
```

EXAMPLE

```
const { register, handleSubmit } = useForm<FormData>({ resolver: zodResolver(schema) });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bank KYC form with checks done by machine at each box; React Hook Form validates efficiently.

தமிழ்

வங்கி KYC படிவத்தில் ஒவ்வொரு கட்டத்திலும் இயந்திரம் சரிபார்ப்பது போல React Hook Form திறமையாக சரிபார்க்கிறது.

PROS

- Fewer re-renders
- Schema validation with Zod

CONS

- Library learning curve

ADVANCED / NEW FEATURES

- Native `<form action>` support in React 19

18. Server State with TanStack Query

DEFINITION

Caching, refetching, pagination and mutations for API data with minimal code.

SYNTAX

```
const { data } = useQuery({ queryKey, queryFn });
```

EXAMPLE

```
const { data, isLoading } = useQuery({ queryKey: ['users'], queryFn: fetchUsers });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A smart cold-storage: remembers what you fetched, refreshes it when stale; TanStack Query caches server data.

தமிழ்

நீங்கள் எடுத்ததை நினைவில் வைத்து பழையதானால் புதுப்பிக்கும் குளிர்பதன கிடங்கு போல TanStack Query சேவையகத் தரவைக் கேச் செய்கிறது.

PROS

- Caching, retries, background refresh
- Less boilerplate

CONS

- Extra dependency
- Query-key discipline needed

ADVANCED / NEW FEATURES

- TanStack Query v5 with Suspense support

19. Testing React

DEFINITION

Test behaviour with Vitest/Jest and React Testing Library; mock APIs with MSW; end-to-end with Playwright.

SYNTAX

```
render(<A />); await user.click(btn);
```

EXAMPLE

```
render(<Counter />);
await userEvent.click(screen.getByRole('button'));
expect(screen.getByText('1')).toBeInTheDocument();
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A mock drill: practise a fire drill without a real fire; tests simulate users safely.

தமிழ்

உண்மையான தீ இல்லாமல் தீ ஒத்திகை செய்வது போல tests பயனர்களைப் பாதுகாப்பாக உருவகப்படுத்துகின்றன.

PROS

- Confidence when changing code
- Documentation by examples

CONS

- Time to write
- Over-mocking makes tests brittle

ADVANCED / NEW FEATURES

- Vitest + Testing Library
- Playwright component testing

Master Level

20. React 19 Features

DEFINITION

Actions, `useActionState`, `useOptimistic`, `use()` for promises, Server Components and the React Compiler reduce boilerplate and manual memoisation.

SYNTAX

```
const [s, action] = useActionState(fn, init);
```

EXAMPLE

```
const [state, formAction] = useActionState(saveUser, null);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A new self-service kiosk that handles the whole form in one go; React 19 Actions simplify submissions and pending states.

தமிழ்

முழுப் படிவத்தையும் ஒரே முறையில் கையாளும் புதிய சுய-சேவை கியோஸ்க் போல React 19 Actions சமர்ப்பிப்பு, pending நிலைகளை எளிதாக்குகின்றன.

PROS

- Less boilerplate for forms
- Built-in pending states

CONS

- New concepts to learn
- Needs framework support for server parts

ADVANCED / NEW FEATURES

- React 19, 19.2 (`<Activity>`, `useEffectEvent`)
- React Compiler 1.0

21. Project Architecture and Best Practices

DEFINITION

Feature folders, absolute imports, error boundaries, accessibility, environment configs, linting and CI.

SYNTAX

```
src/features/x/
```

EXAMPLE

```
src/
  features/auth/
  features/products/
  components/ui/
  lib/
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A mall organised by floors and sections so customers and staff can find things; feature folders organise code.

தமிழ்

வணிக வளாகம் தளங்களாகவும் பிரிவுகளாகவும் ஒழுங்கமைக்கப்பட்டிருப்பது போல feature folders குறியீட்டை ஒழுங்குபடுத்துகின்றன.

PROS

- Easier maintenance
- Team scaling

CONS

- No official structure
- Over-engineering risk

ADVANCED / NEW FEATURES

- Framework-led structure (Next.js, React Router v7)

22. Real Project: E-commerce Front End

DEFINITION

Product list, search, filters, cart (context/Zustand), checkout form, auth routes, TanStack Query for API, deployed on Vercel/Netlify.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A neighbourhood store's online shop: product list, cart, checkout - the same steps as any e-commerce site.

தமிழ்

அக்கம்பக்கக் கடையின் ஆன்லைன் கடை: பொருள் பட்டியல், கூடை, பணம் செலுத்துதல் - எந்த இ-காமர்ஸ் தளத்தின் அதே படிகள்.

PROS

- Portfolio-worthy project
- Practises most React skills

CONS

- Needs backend/API

ADVANCED / NEW FEATURES

- Use Server Components/Next.js for production

23. Deployment and Interview Prep

DEFINITION

Build with `npm run build`, deploy static files, set env vars, and practise explaining rendering, reconciliation and hooks rules.

SYNTAX

```
npm run build
```

EXAMPLE

```
npm run build
npx serve dist
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Packing the finished product and delivering it to customers; `build` makes optimised files you host anywhere.

தமிழ்

தயாரான பொருளைப் பேக் செய்து வாடிக்கையாளருக்கு அனுப்புவது போல `build` எங்கும் host செய்யக்கூடிய உகந்த கோப்புகளை உருவாக்கும்.

PROS

- Static hosting is cheap
- Easy CI/CD

CONS

- SPA needs rewrite rules
- SEO needs SSR for some cases

ADVANCED / NEW FEATURES

- Vercel/Netlify one-click deploys
- React Server Components with frameworks

Interview Question Bank

Q1. What is the virtual DOM?
An in-memory representation of the UI; React diffs it against the previous version and updates only the changed real DOM nodes.

Q2. State vs props?
Props are inputs passed from the parent (read-only); state is data owned and changed by the component.

Q3. Why are keys needed?
Keys let React identify list items between renders so it can update, move and remove them correctly.

Q4. Rules of hooks?
Call hooks only at the top level of components/custom hooks, never inside loops, conditions or nested functions.

Q5. useEffect dependency array?
Controls re-running: `[]` once, `[a]` when `a` changes, none after every render.

Q6. Controlled vs uncontrolled components?
Controlled inputs are driven by state; uncontrolled keep their value in the DOM, read via refs.

Q7. What is prop drilling and how to avoid it?
Passing props through many levels; use Context, composition or a state library.

Q8. `useMemo` vs `useCallback`?
`useMemo` memoises a computed value; `useCallback` memoises a function.

Q9. What is reconciliation?
The algorithm React uses to compare element trees and compute minimal updates.

Q10. What are Server Components?
Components rendered on the server that send no JavaScript to the client and can access data directly.

Capstone Project Ideas

1. Todo app with filters
2. Movie search app with TanStack Query
3. Kanban board with drag and drop
4. Blog with React Router and Markdown
5. E-commerce storefront